설비 고장을 예측하는 AI 데이터 분석

Pandas 기초

Pandas 입문과 데이터 불러오기

posco

CONTENTS

목차

01 Pandas 입문과 표 데이터의 구조

02 CSV 불러오기와 read_csv 옵션

03 옵션을 함께 쓰기

posco

왜 데이터분석에 도메인 지식이 필요한가

같은 숫자를 정상으로도 이상으로도 만드는 것이 설비에 대한 이해

01 · 같은 숫자, 다른 판정

오일온도 75도는 정상인가

이 압축기에서는 관측된 최댓값이 75.0도로, 정상 범위의 위쪽 끝. 같은 75도라도 다른 설비였다면 이미 과열. 숫자만으로는 정상과 이상을 가를 수 없음.

02 · 이상해 보이는 값의 이유

꺼져 있는 시간이 절반

전류가 0에 가까운 줄이 절반 넘게 섞여 있음. 설비가 켜졌다 꺼졌다 한다는 것을 알면 계산 오류가 아니라 정상 동작. 모르면 데이터가 잘못됐다고 결론 내리고 멀쩡한 값을 버리게 됨.

03 · 무엇을 볼지 정해 줌

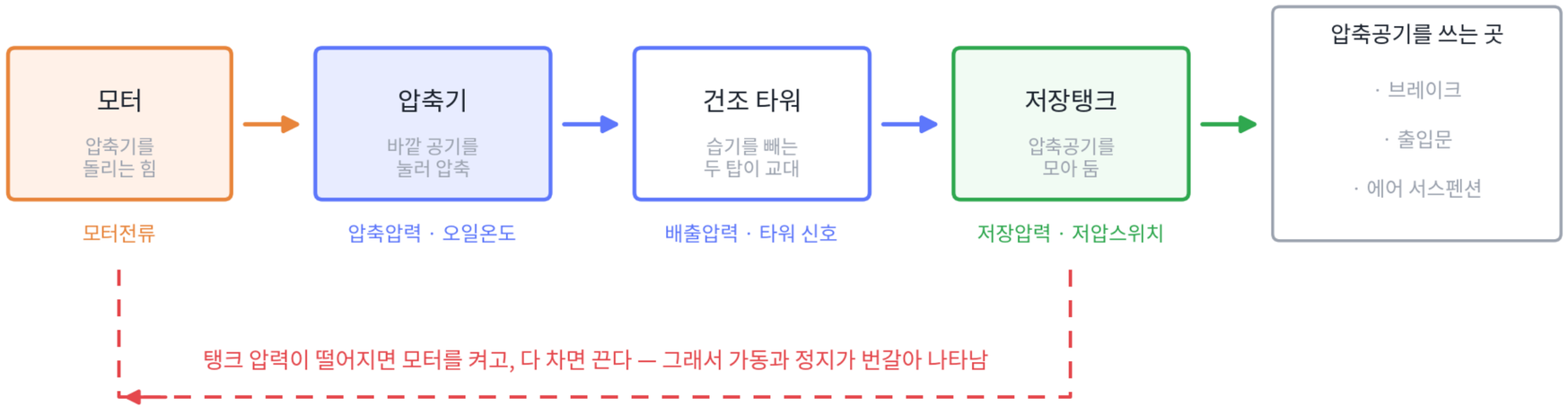
볼 곳을 좁혀 주는 지도

탱크 압력이 떨어질 때 모터가 켜진다는 사실을 알면 압력과 전류를 짝으로 봐야 한다는 판단이 나옴. 도메인이 분석의 출발점을 정해 줌.

분석 대상 설비 — 지하철 공기압축기(MetroPT-3)

압축공기를 만들어 브레이크·출입문·서스펜션을 움직이는 장치 —모터가 공기를 눌러 저장탱크에 모아 둠

지하철 공기압축기(APU) — 압축공기를 만들어 브레이크와 문을 움직인다



센서가 일정 시간마다 값을 한 줄씩 기록 — 그 한 줄이 데이터 한 행이 된다

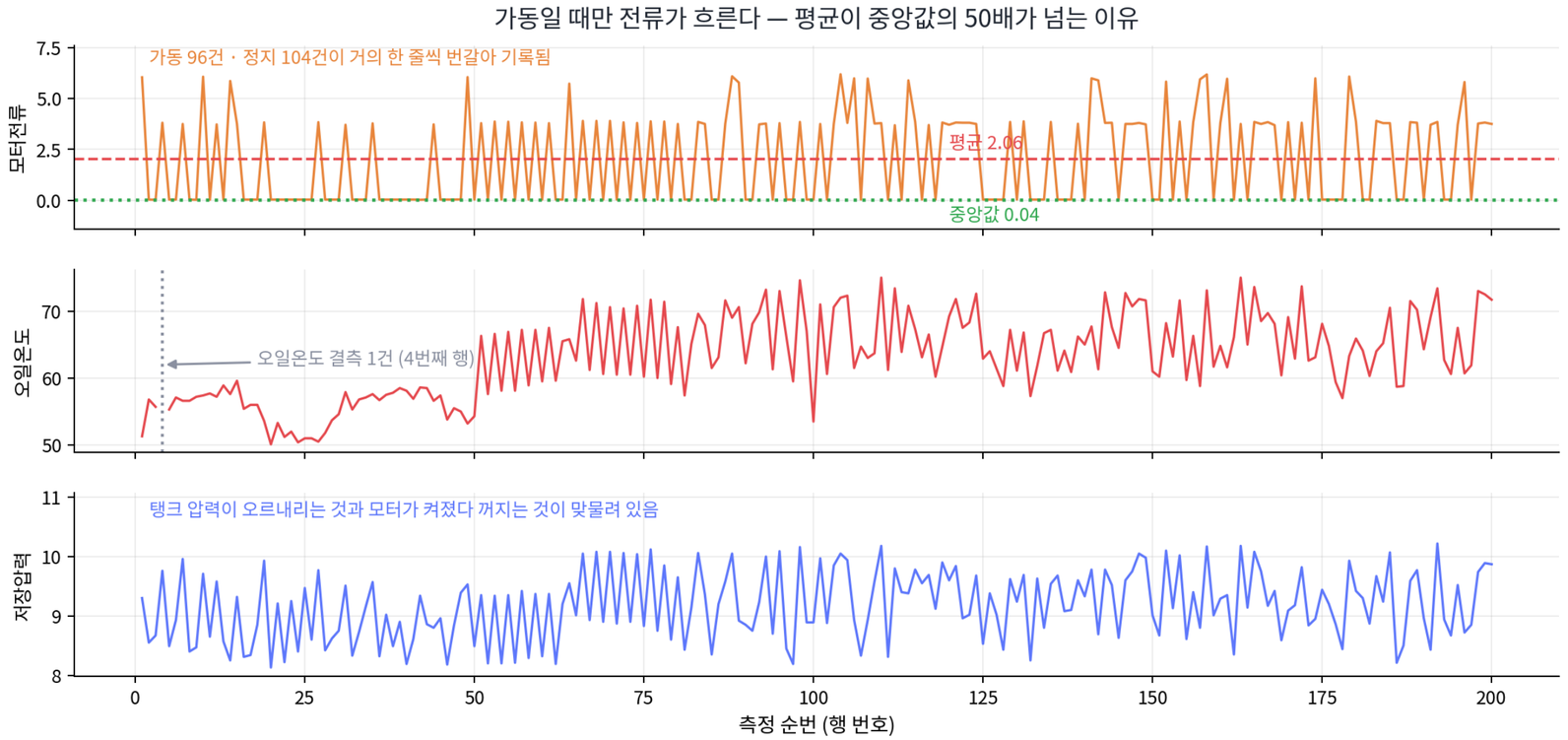
핵심 변수 — 측정시각 · 압축·배출·저장 압력(bar) · 오일온도(℃) · 모터전류(A) · 가동상태(가동/정지)

정상과 이상 — 가동하면 모터전류가 오르고 압력이 차오름 · 오일온도가 비정상적으로 오르거나 압력이 안 차오르면 고장 신호

출처: MetroPT-3 지하철 공기압축기 데이터셋(UCI #791, CC BY 4.0) 발췌 · 200행 7열 — 시각·문자·숫자가 한 표에 섞여 있음

가동과 정지가 번갈아 나타나는 데이터

같은 설비인데 켜졌을 때와 꺼졌을 때의 값이 전혀 다름 — 통계를 읽을 때 반드시 기억할 것



모터전류는 가동일 때만 흐름 — 200줄 중 96줄이 가동, 104줄이 정지

오일온도에 빈 칸이 하나 있음 — 센서가 그 시각 값을 못 남긴 자리

가동상태 열에 가동·정지가 글자로 적혀 있어, 통계가 이상하게 나올 때 이유를 확인할 수 있음

posco

01

Pandas 입문과 표 데이터

표를 코드로 다루는 이유 · Series와 DataFrame · 행 · 열 · 인덱스

posco

배열로는 왜 부족한가

앞 단원의 배열은 숫자만 담을 수 있고 열에 이름이 없음

Q1 설비 로그를 배열에 담아 보면

측정시각과 가동상태는 글자 — 숫자 배열에 같이 담기지 않음

Q2 열을 고를 때도 번호뿐

네 번째 열이 무엇이었는지 매번 기억하거나 세어 봐야 함

Q3 DataFrame은 둘 다 해결

글자와 숫자를 한 표에, 열은 이름으로 — 그래서 코드가 짧아짐

posco

설비 데이터가 표로 쌓이는 방식

행은 한 시점의 측정 기록, 열은 하나의 측정 항목

↓ 열 (측정 항목)

index	측정시각	오일온도	모터전류	가동상태
0	06:38:47	51.3	6.04	가동
1	07:28:21	56.8	0.04	정지
2	08:17:54	55.7	0.03	정지
3	09:07:27	NaN	3.81	가동
4	09:57:01	55.3	0.04	정지

인덱스 (행 번호표) →

행 → 한 시점 기록

행은 시간이 지나며 늘어나고, 열은 측정 항목 수만큼 고정 — 다른 설비 데이터도 같은 구조

posco

Series — 한 줄짜리 데이터

Pandas의 두 그릇 중 작은 그릇 — 표의 한 열에 해당

01 · 정의

Series란

같은 종류 값이
일렬로 늘어선 한 열

온도 열 전체처럼
한 가지 의미의 값이 순서대로 모인 것

표의 한 조각 — **한 열**

02 · 인덱스

각 값에 붙는 번호표

0, 1, 2 순서로
번호 부여

출석부 학생 번호에 점수가 적힌 것처럼,
번호로 원하는 값을 바로 찾기

빠른 접근의 **키**

03 · 활용

DataFrame과의 관계

DataFrame의
열 한 개가 Series

따로 외울 개념이 아니라,
우리가 늘 다루는 표의 한 조각

열 추출 결과 = Series

posco

DataFrame — 표 전체를 담는 그릇

여러 Series가 모이면 우리가 아는 엑셀 표 — 그것이 DataFrame

01 · 정의

DataFrame이란

여러 Series가 옆으로 모인 표

설비 데이터 7개
열이 옆으로 붙으면 하나의 DataFrame

엑셀에서 보던 그 **표**

02 · 강점

무엇이 가능한가

원하는 행·열을 자유롭게 선택

온도 열만 보기·100도 넘는 행만
골라내기 같은 작업이 간단한 코드로

행·열 단위 **조작**

03 · 핵심

CSV와의 관계

CSV를 불러오면 바로 DataFrame

앞으로 df라고 쓰면
거의 항상 이 DataFrame을 가리킴

분석 **중심 객체**

표를 다룰 때 쓰는 행·열·인덱스 단어 정리로 이어짐

행 · 열 · 인덱스 개념 잡기

행은 가로 한 건, 열은 세로 한 항목, 인덱스는 행 번호표

ROW · 행

가로 한 줄 — 특정 시점의 한 건 기록

그 시각의 오일온도·모터전류·압력이 함께 담김

COLUMN · 열

세로 한 칸 — 하나의 측정 항목

오일온도면 오일온도값만, 모터전류이면 모터전류값만 죽 들어감

INDEX · 인덱스

각 행에 붙은 번호표 — 0부터 시작

엑셀의 왼쪽 행 번호와 같은 역할

idx	측정시각	오일온도	모터전류
0	06:38:47	51.3	6.04
	07:28:21	56.8	0.04
	08:17:54	55.7	0.03
	09:07:27	NaN	3.81
	09:57:01	55.3	0.04

노랑 = 행(row) — 한 시점 기록 1건

초록 = 열(column) — 측정 항목 1개

파랑 = 인덱스(index) — 행 번호표

가로로 자르면 행, 세로로 자르면 열, 왼쪽 끝 번호가 인덱스

posco

Jupyter에서 DataFrame 출력 읽기

셀에 변수만 적으면 표가 자동 정렬 — 생략 점·NaN 표시도 익히기

01 · 출력 방법

print 없이도 가능

셀에 변수 이름만 df 쓰고 실행

표가 깔끔하게 정렬되어 자동 출력, print를 쓸 필요 없음

Jupyter의 **기본 동작**

02 · 가운데 생략

행이 많으면

가운데가 ...로 줄어들어 보임

위·아래 5줄만 보이고 가운데는 점 세 개로 표시, 데이터는 그대로

보기 좋게 **화면만 축약**

03 · 빈 칸 표시

값이 없는 칸

NaN 으로 표시되는 결측치

Not a Number → 측정값이 빠진 자리, 뒤에서 자세히 다룸

결측치 **전용 표기**

이제 전체 작업이 어떤 순서로 흘러가는지 큰 그림 그리기

posco

Pandas 데이터 분석 4단계 작업 흐름

불러오기 → 미리보기 → 구조 확인 → 통계 요약, 4단계의 흐름

STEP 1

불러오기

CSV 파일을 read_csv로 읽어
DataFrame으로 변환

문 열고 들어가기

→

STEP 2

미리보기

head·tail로
실제 값을 직접 눈으로 확인

방 들여다보기

→

STEP 3

구조 확인

행·열 수,
자료형 같은 데이터 뼈대 확인

구조 둘러보기

→

STEP 4

통계 요약

describe로
평균·최대·최소를 한 번에 추출

평수 확인

새 데이터를 만나면 늘 이 순서 — 베테랑 분석가도 예외 없이 이 네 단계를 밟음

posco

02

CSV 불러오기와 read_csv 옵션

경로 · 인코딩 · 구분자 · 열 선택 — 실제로 막히는 네 가지

posco

지금까지 파일을 열던 방법

08 파트의 csv 모듈도, 11 파트의 배열도 표를 표로 다루지는 못했음

01 • 08 파트 방식

한 줄씩 리스트로

with open으로 열고 csv.reader로 한 행씩 받음.
헤더는 직접 걷어내고, 계산하려면 값마다 float으로 바꿔야 했음.

02 • 11 파트 방식

숫자만 배열로

genfromtxt로 숫자 열만 한 번에 읽음.
계산은 빨라졌지만 글자 열은 담기지 않고 열에 이름도 없음.

03 • 둘 다 남는 일

표 모양은 손으로

헤더 걷어내기·형변환·빈 칸 표시·열 이름 붙이기를 매번 직접.
파일이 바뀌면 그 코드도 다시 씀.

read_csv는 이 네 가지를 함수 하나가 대신해 줌 — 다음 장에서 실제로 얼마나 줄어드는지 비교

같은 열을 세 가지 방법으로 읽어 보면

결과는 같은데 손이 가는 일이 다름 — 무엇이 사라졌는지 보기

CODE

```
F = 'data/12_metro_compressor.csv'      # 오일온도는 왼쪽에서 다섯 번째 열
# 08 파트 방식 — csv 모듈
import csv
oil = []
with open(F, encoding='utf-8') as f:
    reader = csv.reader(f)
    next(reader)                        # 헤더를 직접 걷어냄
    for row in reader:
        oil.append(float(row[4]) if row[4] else None)
# 11 파트 방식 — NumPy 배열
import numpy as np
oil = np.genfromtxt(F, delimiter=',', skip_header=1, usecols=4)
# 12 파트 방식 — Pandas
import pandas as pd
oil = pd.read_csv(F)['오일온도']
```

세 방법 모두 같은 200개 값을 얻지만, 표 모양까지 만들어 주는 것은 read_csv 하나

posco

그럼 실무에서는 무엇을 쓰나

표로 분석할 일이면 pandas — csv 모듈이 더 나은 자리도 분명히 있음

01 • 기본은 pandas

분석 업무의 대부분

표를 읽고 고르고 통계 내는 일이면 거의 언제나 pandas.
엑셀·데이터베이스도 같은 방식으로 읽어 옴.

02 • csv 모듈이 나은 때

한 줄씩 흘려보낼 때

메모리에 다 못 올릴 만큼 큰 로그를 한 줄씩 처리하거나,
표가 아닌 형식을 직접 다룰 때.

03 • 판단 기준

표로 볼 것인가

열 단위로 계산하거나 조건으로 골라낼 것이면 pandas.
한 줄 읽고 버릴 것이면 csv 모듈.

이 과정은 여기서부터 끝까지 pandas — 08 파트에서 배운 open과 csv는 그 아래에서 그대로 돌아감

posco

read_csv가 하는 일

쉼표로 이어진 글자 뭉치를 칸이 나뉜 표로 바꿔 주는 함수

read_csv가 하는 일 — 쉼표로 이어진 글자를 칸이 나뉜 표로 바꾼다

CSV 원본 — 메모장으로 열면 이렇게 보임

```
측정시각,압축압력,오일온도,가동상태
2020-02-27 06:38:47,9.3,51.3,가동
2020-02-27 07:28:21,8.55,56.8,정지
2020-02-27 08:17:54,8.67,55.7,정지
```

첫 줄은 열 이름(헤더), 그 아래가 데이터 행

쉼표가 칸을 나누는 기준 — 이 기준이 어긋나면 표가 무너짐

read_csv

DataFrame — 파이썬 안에서 다룰 수 있는 표

	측정시각	압축압력	오일온도	가동상태
0	06:38:47	9.30	51.3	가동
1	07:28:21	8.55	56.8	정지
2	08:17:54	8.67	55.7	정지

맨 왼쪽 0 · 1 · 2 는 파일에 없던 인덱스 — read_csv가 붙여 줌

첫 줄은 열 이름(헤더), 그 아래가 데이터 행 — 쉼표가 칸을 나누는 기준

왼쪽 0 · 1 · 2 는 파일에 없던 번호 — read_csv가 인덱스로 붙여 줌

이 기준이 어긋나면 표가 무너짐 — 그래서 구분자 옵션이 필요해짐

posco

pd.read_csv() 기본 사용법

파일 이름을 따옴표로 감싸 넣으면 결과가 DataFrame으로 반환

NOTEBOOK · CELL 2

```
import pandas as pd     # 앞 단원의 np 와 같은 별명 약속

# CSV를 DataFrame으로 — 분석의 시작
df = pd.read_csv("data/12_metro_compressor.csv")
df.head()
```

앞 단원에서는 열 번호와 구분자를 직접 지정해 숫자만 읽었지만, read_csv는 한 줄로 표 전체를 읽어 옴

열 이름·글자 열까지 그대로 살아 있는 것이 배열과 가장 다른 점

posco

변수에 담기 — df 관례

DataFrame의 줄임말 — 가장 흔히 쓰이는 변수 이름

01 • df 관례

가장 흔한 변수 이름

DataFrame의 줄임말 df

pd와 마찬가지로 의무는 아니지만 모두 이렇게 사용

글로벌 **관용 표기**

02 • 여러 데이터

의미를 붙여 구분

df_comp
df_dig

여러 파일을 동시에 다루면
헷갈리지 않게 의미를 붙여 구분

역할별 **접미사**

03 • 재사용

모든 작업의 출발점

한 번 담으면 df로 모든 작업

head로 보고, shape로 재고, describe로 뽑고
모두 df에서 출발

분석의 **중심점**

담아 두면 head로 보고, shape로 재고 —모든 작업이 여기서 출발

posco

실습 1. 설비 센서 파일 불러와 확인하기

압축기 데이터를 읽어 크기와 앞부분을 눈으로 확인

목표

파일을 불러와 몇 줄 몇 칸인지 확인하고 앞부분을 화면에 띄우기

단계

① Pandas를 약속된 별명으로 불러오기

② 데이터 폴더에 있는 압축기 파일을 읽어 변수에 담기

③ 담은 것이 몇 줄 몇 칸인지 확인해 출력

④ 앞부분 몇 줄을 화면에 띄워 실제 값 보기

⑤ 빈 칸으로 보이는 자리가 있는지 눈으로 찾아보기

예상 결과

200행 7열 · 앞 5줄이 표로 출력 · 오일온도 열 한 자리가 비어 있음

posco

실습 1. 설비 센서 파일 불러와 확인하기 결과 해석

불러오기와 확인은 한 세트 —읽었다고 끝이 아니다

01 • 변수에 담아야 하는 이유

안 담으면 한 번 보고 사라짐

읽기만 하면 화면에 한 번 나오고 끝.
다음 줄에서 다시 쓸 수 없음.
그래서 반드시 이름을 붙여 담아 두는 것이 습관.

02 • 인덱스는 어디서 왔나

파일에 없던 번호

맨 왼쪽 0 · 1 · 2 는 원본 파일에 없음.
read_csv가 행마다 붙여 준 번호표.
그래서 200줄이면 마지막 번호는 199.

03 • 검산

200행 7열

메모장으로 열면 제목 줄 포함 201줄.
제목 줄이 열 이름으로 올라가
데이터는 200줄이 됨.
열은 제목 줄의 쉼표 개수보다 하나 많음.

실습 1. 설비 센서 파일 불러와 확인하기 정답

읽은 결과는 반드시 변수에 담아야 다음 줄에서 쓸 수 있다

CODE

```
import pandas as pd

# 출처: MetroPT-3 지하철 공기압축기 공개 데이터셋 발췌
df = pd.read_csv('data/12_metro_compressor.csv')

print(df.shape)     # (200, 7)
print(df.head())
# 오일온도 열의 3번 행이 NaN — 값이 없는 자리
```

posco

앞 단원에서 쓰던 설정이 read_csv에서는 이렇게

이름만 바뀐 것이 대부분 —하나는 완전히 달라짐

01 • 이름만 바뀐 것

delimiter → sep

값을 나누는 기호를 알려 주는 설정.
이름만 sep 으로 바뀜.
쉼표가 아닌 파일을 만났을 때 쓰는 것도 그대로.

02 • 그대로인 것

encoding • usecols

글자를 읽는 방식을 정하는 encoding 은
파일을 열 때 쓰던 것과 같음.
필요한 열만 고르는 usecols 도 이름이 같음.

03 • 완전히 달라진 것

열을 이름으로 고름

앞에서는 usecols 에 열 번호를 넣어야 했음.
read_csv 는 열 이름을 그대로 넣음
몇 번째 열인지 세지 않아도 됨.

경로가 틀리면 나는 오류와 점검 순서는 앞에서 배운 그대로 — read_csv도 다르지 않음

posco

실습 2. 필요한 열만 이름으로 골라 읽기

열이 많은 파일에서 쓸 열만, 행이 많으면 앞부분만

목표

열을 이름으로 골라 읽었을 때와 행 수를 제한했을 때의 크기 변화를 확인

단계

① 압축기 파일에서 측정시각·오일온도·모터전류 세 열만 이름으로 골라 읽기

② 골라 읽은 것의 크기를 출력해 열이 몇 개로 줄었는지 확인

③ 같은 파일을 앞에서 스무 줄만 읽어 변수에 담기

④ 그 크기도 출력해 행이 몇 개로 줄었는지 확인

⑤ 앞 단원처럼 번호로 골랐다면 무엇이 불편했을지 말해 보기

예상 결과

열을 고르면 200행 3열, 스무 줄만 읽으면 20행 7열

posco

실습 2. 필요한 열만 이름으로 골라 읽기 결과 해석

무엇을 줄였는지는 크기의 어느 숫자가 변했는지로 확인

01 • 열을 고르면

뒤 숫자가 줄어듦

200행 7열이 200행 3열로.
줄 수는 그대로이고 칸 수만 줄어듦.
필요한 항목만 남기는 것이므로
행에는 영향이 없음.

02 • 행을 제한하면

앞 숫자가 줄어듦

200행 7열이 20행 7열로.
항목은 그대로이고 줄 수만 줄어듦.
큰 파일의 생김새를 빠르게 훑어볼 때 씀.

03 • 번호 대신 이름

순서가 바뀌어도 안전

번호로 고르면 열 순서가 바뀌었을 때
조용히 다른 열을 가져옴.
이름으로 고르면 없는 이름이라는
오류가 나므로 바로 알아챔.

지금 파일은 작아 차이를 못 느끼지만, 수백만 행짜리에서는 이 습관이 시간을 크게 줄임

실습 2. 필요한 열만 이름으로 골라 읽기 정답

열을 고르면 뒤 숫자가, 행을 제한하면 앞 숫자가 줄어든다

CODE

```
import pandas as pd

F = 'data/12_metro_compressor.csv'

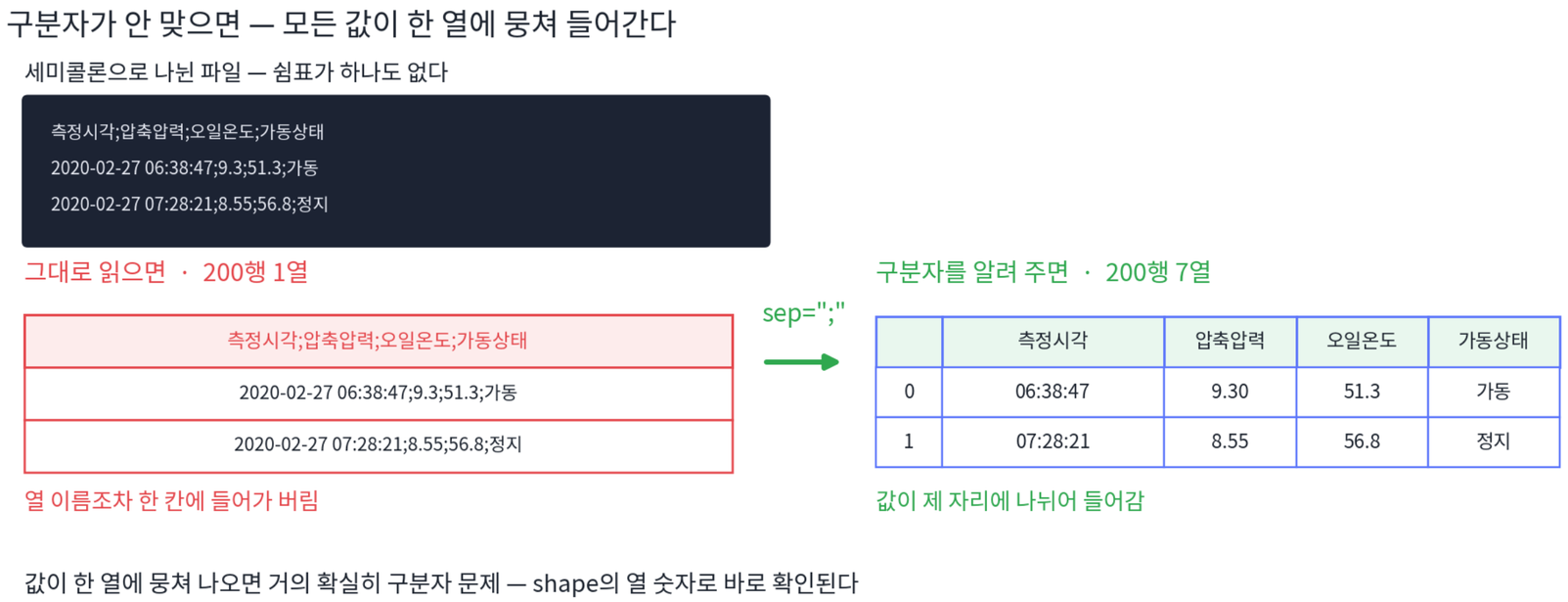
# 번호가 아니라 이름으로 고름
sel = pd.read_csv(F, usecols=['측정시각', '오일온도', '모터전류'])
print(sel.shape)    # (200, 3)

part = pd.read_csv(F, nrows=20)
print(part.shape)   # (20, 7)

# 시도) 이름을 한 글자만 틀리면 없는 열이라는 오류가 남
# pd.read_csv(F, usecols=['오일 온도'])   # ValueError 발생
```

구분자가 안 맞으면 어떻게 되는가

쉼표가 아닌 기호로 나뉜 파일을 그대로 읽으면 값이 한 열에 뭉친다



값이 전부 한 칸에 들어가 200행 1열이 됨 — 열 이름조차 한 칸에 붙어 버림

구분자를 알려 주면 제자리에 나뉘어 200행 7열이 됨

shape의 열 숫자가 1이면 거의 확실히 구분자 문제 — 가장 빨리 알아채는 방법

posco

실습 3. 구분자와 인코딩 맞춰 읽기

세미콜론으로 나뉜 파일을 제대로 읽히게 만들기

목표

그냥 읽었을 때와 구분자를 알려 줬을 때의 크기를 비교

단계

① 세미콜론으로 나뉜 파일을 아무 설정 없이 읽어 변수에 담기

② 몇 줄 몇 칸인지 출력해 무엇이 잘못됐는지 확인

③ 값을 나누는 기호가 무엇인지 알려 주고 다시 읽기

④ 다시 읽은 것의 크기를 출력해 제대로 나뉘었는지 확인

⑤ 한글 열 이름이 깨지지 않고 나오는지도 함께 확인

예상 결과

그냥 읽으면 200행 1열, 구분자를 알려 주면 200행 7열

posco

실습 3. 구분자와 인코딩 맞춰 읽기 결과 해석

행은 맞는데 열이 하나 —이 조합이 구분자 문제의 신호

01 • 증상 읽는 법

행 200 • 열 1

줄은 제대로 세었는데 줄 안을 못 나눈 상태.
행 수가 맞으면서 열이 1이면
거의 언제나 구분자 문제.

02 • 무엇으로 나뉘었나

메모장으로 열어 보기

찍어서 맞히는 것보다
원본을 직접 여는 편이 확실함.
쉼표인지 세미콜론인지 탭인지 눈으로 바로 보임.

03 • 인코딩은 다른 문제

글자가 깨지면

칸은 제대로 나뉘었는데 한글이 깨져 보이면 인코딩 문제.
utf-8과 cp949를 하나씩 시도해 맞는 쪽을 찾음.

posco

실습 3. 구분자와 인코딩 맞춰 읽기 정답

무엇으로 나뉘었는지 알려 주는 것만으로 표가 제자리를 찾는다

CODE

```
import pandas as pd

F = 'data/12_metro_compressor_semicolon.csv'

# 그냥 읽으면 한 열에 뭉침
a = pd.read_csv(F)
print(a.shape)      # (200, 1)

# 값을 나누는 기호를 알려 주면 제자리에 나뉨
b = pd.read_csv(F, sep=';', encoding='utf-8')
print(b.shape)      # (200, 7)
print(b.columns.tolist())

# 인코딩이 안 맞으면 이름이 다른 오류가 남
# pd.read_csv(F, sep=';', encoding='cp949')  # UnicodeDecodeError 발생
```

행 수 200은 두 경우 모두 같음 — 달라지는 것은 열 숫자

posco

엑셀 파일 읽기와 결과 저장

현업 파일은 절반이 엑셀 —읽는 법은 거의 같고, 결과는 파일로 남긴다

CODE

```
import pandas as pd

# 엑셀 파일도 거의 같은 방식 – 시트 이름만 더 지정
# x = pd.read_excel('data/설비로그.xlsx', sheet_name='2월')

# 분석한 결과를 파일로 남기기
df = pd.read_csv('data/12_metro_compressor.csv')
sel = df[['측정시각', '오일온도', '모터전류']]

sel.to_csv('out_selected.csv', index=False, encoding='utf-8-sig')
# index=False         붙여 준 번호는 저장하지 않음
# encoding='utf-8-sig'  엑셀로 열어도 한글이 안 깨짐
```

읽기로 시작해 저장으로 끝나야 분석이 완결됨 — 화면 출력만으로는 남는 것이 없음

03

옵션을 함께 쓰기

지금까지 배운 네 가지 옵션을 한 번에 적용해 보기

실습 4. 옵션을 한 번에 적용해 읽기

구분자 •인코딩 •열 선택 •행 제한을 한 줄에 함께 쓰기

목표

네 가지 옵션을 함께 적용해 원하는 모양으로 읽어 오기

단계

① 세미콜론으로 나뉜 파일을 대상으로 정하기

② 값을 나누는 기호와 글자를 읽는 방식을 함께 알려 주기

③ 측정시각·오일온도·모터전류 세 열만 고르기

④ 앞에서 쉰 줄만 읽도록 제한하기

⑤ 크기와 앞부분을 출력해 의도한 모양으로 나왔는지 확인

예상 결과

50행 3열 • 앞부분에 세 열만 깔끔하게 출력

실습 4. 옵션을 한 번에 적용해 읽기 결과 해석

옵션마다 바꾸는 숫자가 정해져 있다

01 · 순서는 상관없음

이름을 붙여 넘기므로

sep·encoding·usecols·nrows는 이름을 붙여 주는 설정이라 쓰는 순서가 달라도 결과가 같음. 파일 이름만 맨 앞에 와야 함.

02 · 하나씩 빼 보기

각 옵션의 역할이 드러남

구분자를 빼면 열이 1, 열 선택을 빼면 7열, 행 제한을 빼면 200행. 빼 보는 것이 가장 확실한 이해 방법.

03 · 줄이 길어지면

괄호 안에서 줄을 바꿔도 됨

옵션이 많아지면 한 줄이 길어짐. 괄호 안에서는 줄을 바꿔도 이어지는 문장으로 읽히므로 보기 좋게 나눠 써도 됨.

posco

실습 4. 옵션을 한 번에 적용해 읽기 정답

괄호 안에서는 줄을 바꿔 써도 이어지는 문장으로 읽힌다

CODE

```
import pandas as pd

F = 'data/12_metro_compressor_semicolon.csv'

c = pd.read_csv(F,
                sep=';',                # 값을 나누는 기호
                encoding='utf-8',       # 글자를 읽는 방식
                usecols=['측정시각', '오일온도', '모터전류'],
                nrows=50)               # 앞에서 50줄만

print(c.shape)      # (50, 3)
print(c.head(3))
```

앞 숫자는 행 제한이, 뒤 숫자는 열 선택이 정한 값